

Project P3 - Add monitoring and alerting to an app

Student: Stefanica Nicoleta Rodica, DMWD

1. Introduction

This project implements an application that simulates Point-of-Sale (POS) traffic to demonstrate the second way of DevOps: feedback loops. The objective is to create a real-time telemetry pipeline that captures application behaviour, processes performance metrics, displays system health, and automatically routes critical alerts to an external communication channel when operational thresholds are exceeded.

The architecture, as seen in Figure 1, consists of four micro-services orchestrated via Docker Compose:

- The POS Simulator (**Node.js/Express**): the core application that exposes transactional endpoints and instruments business and technical telemetry;
- **Prometheus**: the time-series database that pulls metrics from the application and evaluates alerting rules;
- **Grafana**: real-time dashboards for service health monitoring are provided by this visualisation layer;
- **Alertmanager**: the notification routing gateway that uses webhooks to send active notifications to Discord.

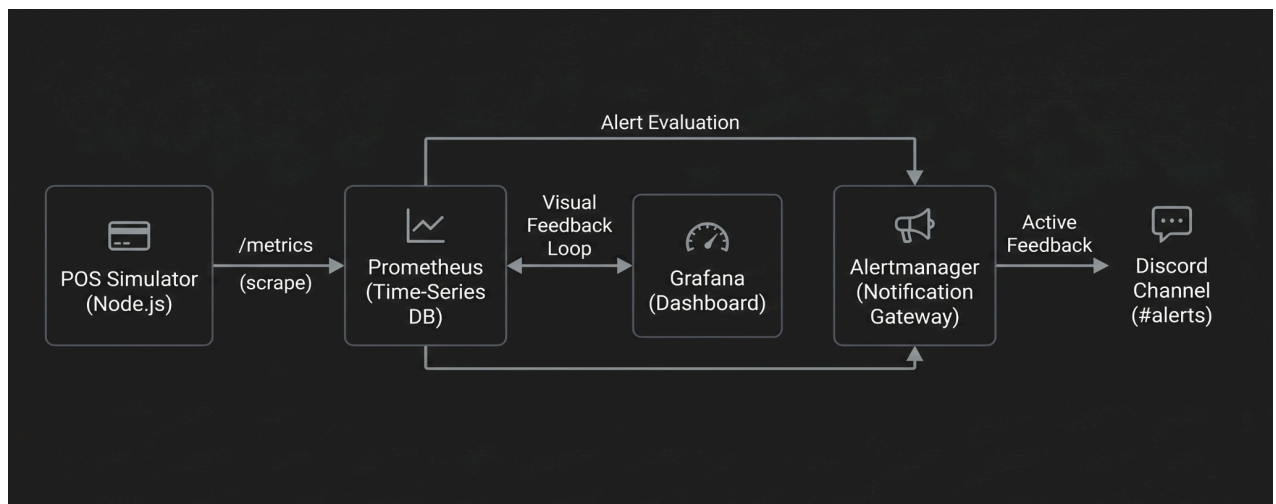


Figure 1: The architecture

2. Metric instrumentation and telemetry

The Node.js application must provide its internal status in order to apply the second way of DevOps (feedback). To achieve this, the application provides a graphical interface used to

simulate realistic Point-of-Sale events, as shown in Figure 1. Every interaction with the buttons generates background telemetry. To capture this data, the prom-client package is embedded within the Node.js runtime to collect metrics and expose them via a standard HTTP endpoint at /metrics in a format that Prometheus can parse.

The application utilizes two main metrics triggered directly by the user interface actions to track technical health and business performance:

- **shop_orders_total** (counter): this metric keeps track of how many transactions the POS system has completed overall (incremented by the "Capture Payment" and "Simulate Bank Timeout" buttons). It utilizes a label named status, which can take the values of success or failed. By separating the counts with labels, Prometheus can determine the precise ratio of unsuccessful payments to total traffic.
- **http_request_latency_seconds** (histogram): this metric calculates how long it takes for incoming HTTP POST requests to reach the checkout endpoints. It makes it possible to identify hidden performance issues (such as the 2-second delay forced by the bank timeout simulator button) before a total system crash occurs.

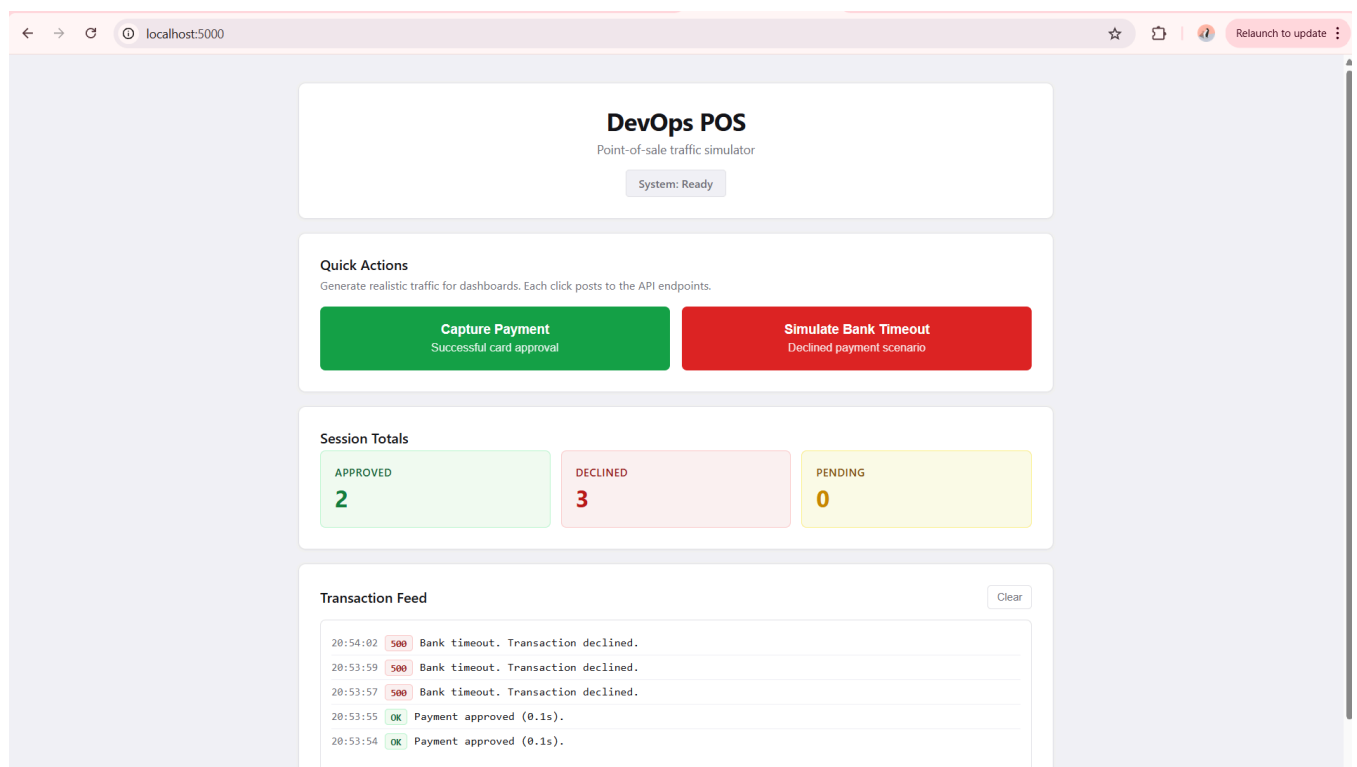


Figure 2: The Node.js application

3. Alerting rules

Telemetry alone is passive; to complete the feedback loop, the system must actively evaluate these metrics against predefined operational thresholds. These boundaries are defined inside Prometheus using PromQL expressions within the alert.yml configuration file.

The system evaluates two critical alerting rules over a 1-minute window:

3.1 CriticalPaymentFailureRate

- **Condition:** This alert triggers if the rate of failed transactions exceeds 15% of the total traffic over the last minute.
- **Logic:** The rule calculates the increase of failed orders divided by the increase of total orders. If the result is greater than 0.15, the alert enters a pending state.
- **Design decisions:** In a standard enterprise environment, scraping intervals and alerting windows are typically set to 5 or 15 minutes to prevent false alarms caused by transient network fluctuations. However, for this laboratory demonstration, the evaluation window was intentionally reduced to 1 minute, combined with a 10-second persistence threshold. This means that once the failure rate crosses the 15% limit, the alert does not fire immediately; instead, it enters a "pending" state and will only transition to "firing" if the condition remains critical for 10 consecutive seconds. This time compression ensures a highly dynamic live presentation, allowing the evaluator to observe the full lifecycle of an incident (pending -> firing -> resolved) without unnecessary delays.

3.2 HighCheckoutLatency

- **Condition:** This alert triggers if the average transaction processing time climbs above 1 second over the last minute.
- **Logic:** Clicking the timeout button forces a 2-second backend delay. Because Prometheus tracks the average latency over a 60-second window, a single slow request might be masked by multiple fast requests (0.1s). The alert will only fire if you click the timeout button multiple times, or if the proportion of slow requests becomes high enough to pull the overall 1-minute average past the 1-second threshold.

4. Grafana visualization

Grafana translates raw data from Prometheus into clean visual indicators. As shown in Figure 3, the dashboard organizes telemetry into three main sections:

- **‘BUSINESS OVERVIEW’**
 - Total successful orders: a trend panel tracking successful payments.
 - Total failed orders: a trend panel tracking failed or timed-out transactions.
- **‘REAL-TIME SERVICE HEALTH’**
 - Average latency (alert threshold: 1s): a radial gauge showing the average processing time.
 - Order failure rate (alert threshold: 15%): a radial gauge showing the percentage of errors.
- **‘TRAFFIC ANALYSIS’**
 - a historical timeline graph displaying the volume of transactions measured in requests per second. It charts the green line (successful orders rate) against

the red line (failed orders rate). During an incident, the red line spikes while the green line flattens out, allowing developers to visualize the exact duration, volume, and severity of the system degradation.

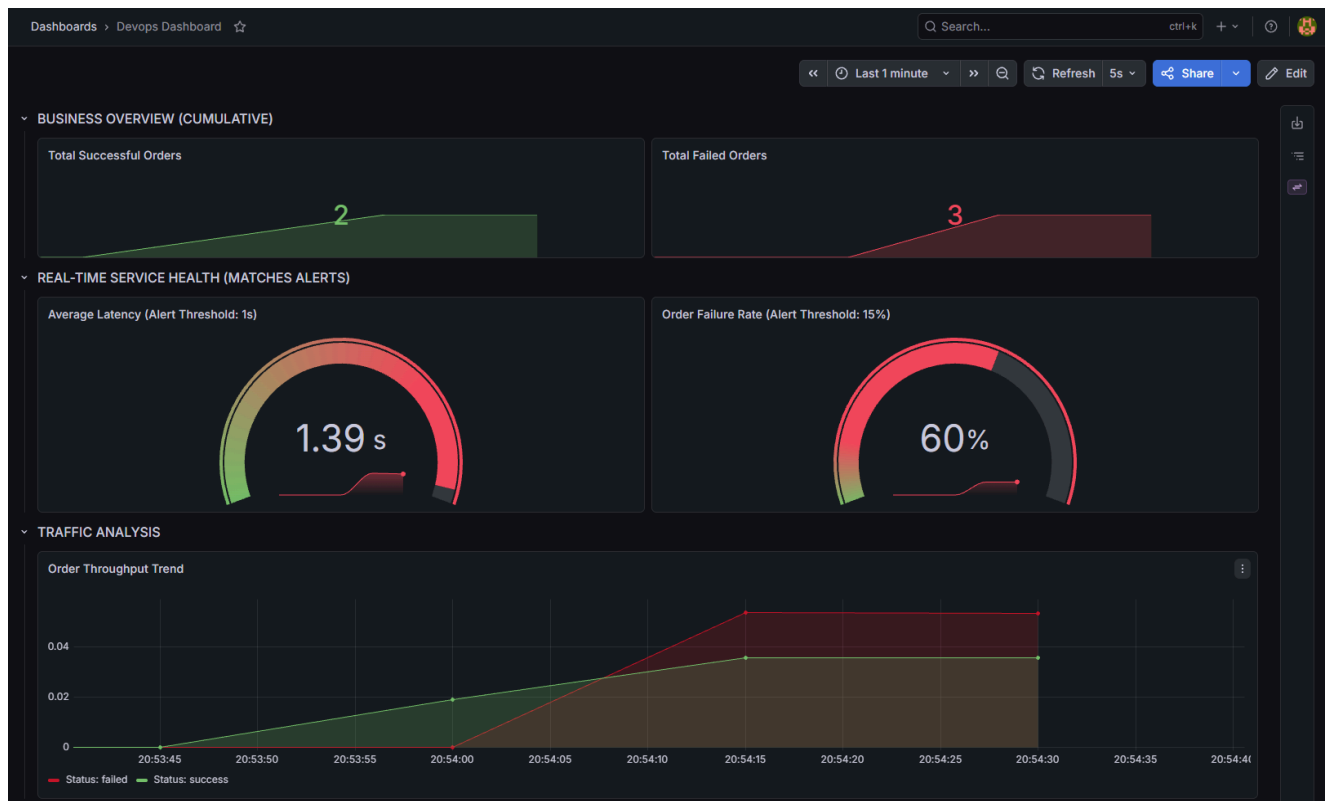


Figure 3: Grafana Monitoring Dashboard

5. Notification routing via Alertmanager and Discord

To complete the active feedback loop of the second way of DevOps, the system must actively notify developers when an incident occurs. This is handled by Alertmanager, which receives active alerts from Prometheus and routes them to an external communication channel.

For this project, a Discord Webhook was configured. When a threshold is crossed, the alert transitions through different states, automatically triggering notifications.

5.1 Prometheus Alert States

Before a notification reaches Discord, Prometheus tracks the life cycle of the alert inside its interface. When a threshold is breached, the alert first enters an orange "Pending" state for 10 seconds to filter out temporary fluctuations. Once this persistence window passes, it transitions into a red "Firing" state and passes the payload to Alertmanager.

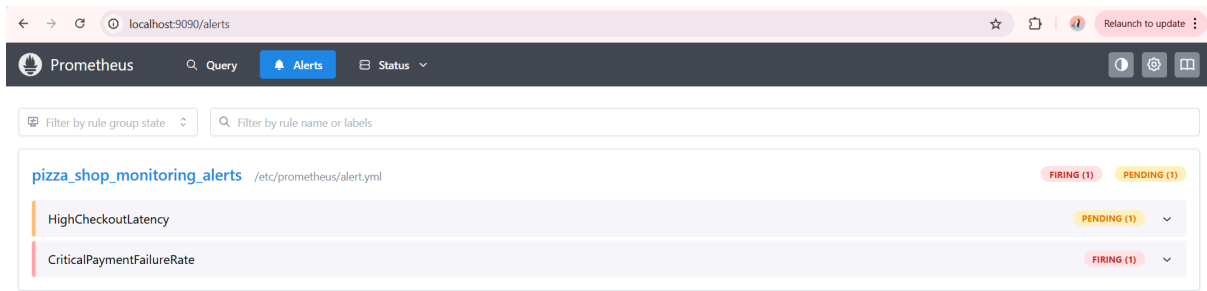


Figure 4: Prometheus interface displaying the alerts and their status

5.2 Discord notifications

Once Alertmanager processes the firing incident, it sends automated, structured messages directly to the team's Discord channel:

- **Incident alert ('FIRING'):** as shown in Figure 5, the team receives an immediate notification containing the alert name, environment details, and an action item prompting them to check the Grafana dashboards.
- **Recovery alert ('RESOLVED'):** once successful traffic drives the metrics back into safe operational zones, Alertmanager automatically sends a resolution message, informing the team that the system health is restored.

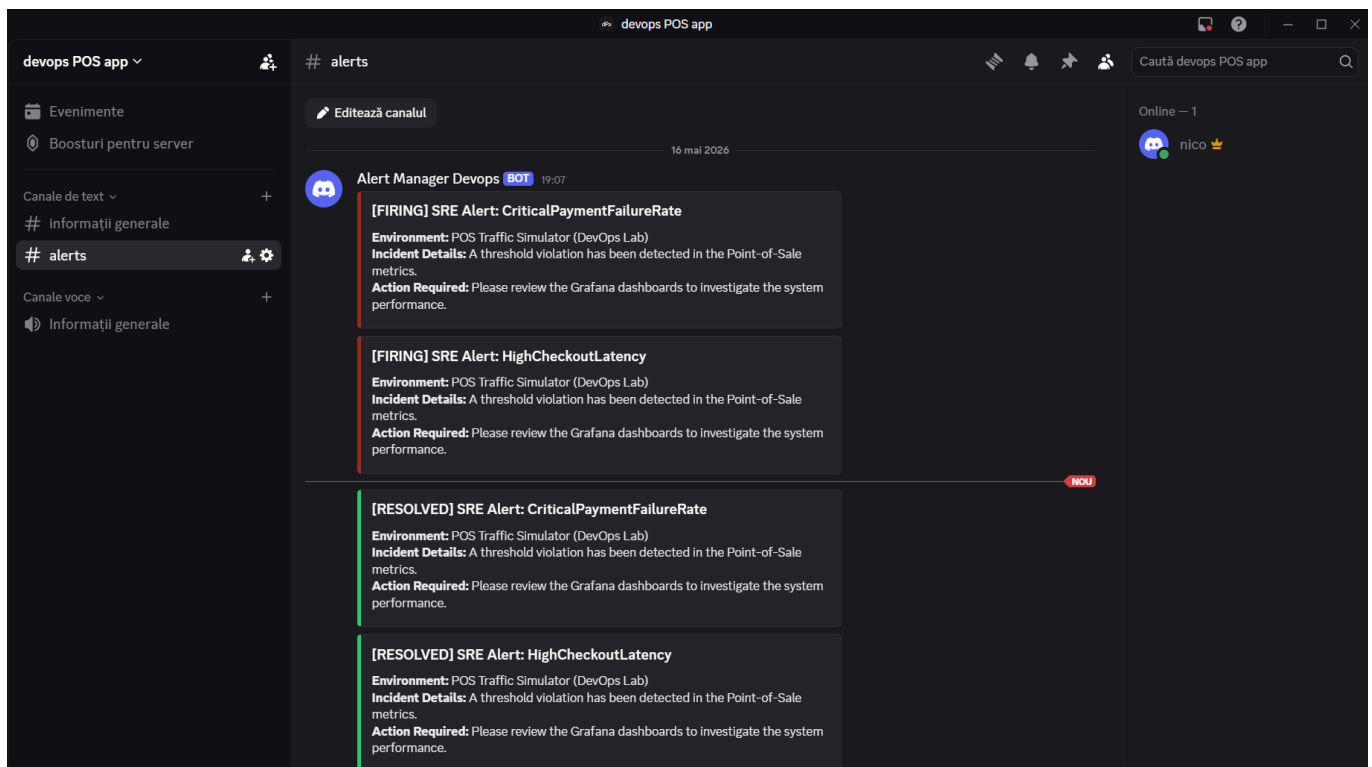


Figure 5: Real-time 'FIRING' and 'RESOLVED' notifications routed to Discord.